

Java OOP Principles - Cheat Sheet

OOP is how Java organizes code around objects instead of just functions. Four ideas hold the whole thing together: encapsulation, inheritance, polymorphism, abstraction.

A simple way to remember them: what state is hidden, what behavior is shared, what behavior changes per type, and what details get skipped at the surface.

1. Encapsulation

Hide the data, expose the behavior. Fields go `private`, and outside code touches them only through methods. This keeps internal state safe and gives you one spot to add validation or change logic later.

```
public class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
        balance += amount;
    }
}
```

Why it matters: - You can swap `balance` from `double` to `BigDecimal` later without breaking callers. - Validation sits in one place (`deposit`), not scattered everywhere.

Common slip: making fields `public` for convenience. Don't. Either expose them through methods or stop exposing them at all.

2. Inheritance

A class can extend another and reuse its fields and methods. Use it when the relationship is genuinely "is-a".

```

public class Animal {
    protected String name;

    public void eat() {
        System.out.println(name + " is eating");
    }
}

public class Dog extends Animal {
    public void bark() {
        System.out.println(name + " barks");
    }
}

```

Dog gets eat() for free.

Things to keep in mind: - Java allows single inheritance only. One class, one parent. - Use `super(...)` to call the parent constructor or method. - A `Car` is not an `Engine`, it has one. If the relationship is “has-a”, use composition instead of `extends`.

3. Polymorphism

Same method name, different behavior based on the object. Java gives you two flavors.

Compile-time (overloading): same name, different parameters.

```

public class Printer {
    public void print(String s) { ... }
    public void print(int i) { ... }
    public void print(String s, int times) { ... }
}

```

Runtime (overriding): child class redefines a parent method.

```

public class Animal {
    public void sound() {
        System.out.println("some sound");
    }
}

public class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("bark");
    }
}

Animal a = new Dog();
a.sound(); // prints "bark"

```

Why it matters: you can write code against the parent type and stop caring which subclass shows up at runtime.

Always use `@Override`. It catches typos at compile time, like writing `sounds()` instead of `sound()`.

4. Abstraction

Show what an object can do, hide how it does it. In Java this shows up as `abstract` classes and `interfaces`.

Abstract class - partial implementation, can hold state.

```

public abstract class Shape {
    abstract double area();

    public void describe() {
        System.out.println("area: " + area());
    }
}

public class Circle extends Shape {
    private double radius;

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

```

Interface - pure contract, no instance state.

```
public interface Payable {  
    double calculatePay();  
}
```

Picking one: - Interface when you only care about what an object can do. - Abstract class when subclasses share state or partial logic. - Since Java 8, interfaces can have default methods. Still no instance fields.

Quick Reference

Concept	Keyword	Mental model
Encapsulation	private , getters/setters	Locked room with a doorman
Inheritance	extends	Child gets parent's traits
Polymorphism	@Override , overloading	One name, many behaviors
Abstraction	abstract , interface	Steering wheel hides the engine

Common Gotchas

- private fields aren't visible to subclasses. Use protected if a subclass needs direct access.
- A class can extend only one class but implement many interfaces.
- Constructors aren't inherited. The child calls the parent via super(...) .
- final on a class blocks inheritance. On a method, it blocks overriding.
- Composition often beats inheritance. If you're extending only to reuse one method, ask whether the class should just hold the other class instead.